New Hampshire MANET Simulator: Methods, Findings, and Validation Status

Ethan Doherty — CS 5976 Directed Study

2026-07-14

Contents

New Hampshire MANET simulator: methods, findings, and validation status

Date: 2026-07-14
Scope: New Hampshire only
Evidence status: working audit report from a dirty, concurrently edited worktree

Executive assessment

The project has produced a useful research prototype, a substantial field-data pipeline, a terrain-aware planning model, and two event-driven network simulators. It has **not** yet produced a field-calibrated digital twin, validated statewide coverage, a proven routing winner, an emergency-ready system, or an accepted Python/Rust replacement-results release.

That distinction is the main conclusion of the audit. There is no repository evidence of deliberate deception. There are, however, important implementation mistakes and overstatements that previously made model outputs look more authoritative than the evidence allowed. The central historical performance, coverage, energy, weather-robustness, and architecture conclusions are withdrawn pending clean replacement runs. The authoritative defect/status index is the audit and correction ledger.

The corrected Rust engine is now reproducible across processes in the tested configuration, but reproducibility is not cross-engine agreement and neither is field validation. The controlled Trial 2 dataset needed for empirical calibration does not yet exist.

1. Original objectives and how they changed

The original course plan is preserved in the weekly execution roadmap. Its success criteria were:

1. Complete architecture and design documentation.
2. Collect at least 2,500 empirical points.
3. produce AIRMap calibration files.
4. Deliver a 10–15 page validation report with RMSE/MAE, predictive-versus-actual heatmaps, and an infrastructure failure matrix.
5. Treat large-scale simulation as an optional future appendix.

The project evolved materially:

Objective	Original intent	Current status and change
Field acquisition	The roadmap targeted two trail campaigns and at least 2,500 empirical points	The planned Tuckerman/Knife-Edge Trial 1 and Huntington/Great Gulf Trial 2 were replaced by the Ammo–summit–Jewell shakedown and a controlled Ammo/Jewell Trial 2. Trial 1 lacks a controlled remote transmitter, authenticated scheduled opportunities, direct-hop identity, and a continuous collector record. The current live-trial quality gate has zero calibration-eligible rows. The unqualified 2,500-point target is superseded by the controlled protocol's per-stratum opportunity requirement.
AIRMap/digital twin	Fit and validate a propagation model from empirical data	The pipeline exists, but no defensible calibration can be fitted from Trial 1. Current ITM, shadowing, receiver, foliage, and body-loss terms remain prospective assumptions.
Validation report	Report held-out RMSE/MAE and predictive-versus-observed results	A true validation report is blocked on Trial 2. Existing Trial 1 maps and residual files cannot fill that role.
Infrastructure assessment	Evaluate whether a relay design closes trail coverage gaps	The later terrain-aware screen contradicted the early free-space picture. The current controlling statewide planning tier fails, and all coverage percentages remain model-to-model screens.
Simulation	Optional future 500-node work	Simulation became a major part of the project: statewide topology, mobility, traffic, energy, weather, kiosk logistics, routing/MAC comparisons, and a Rust acceleration engine. This expansion increased the need for software verification and preregistered replacement analysis.

Objective	Original intent	Current status and change
Geographic scope	Multiple field regions were contemplated during the project	The active study is now **New Hampshire only**. Brenta/Italy is dead and excluded from the analysis, recommendations, and future-work plan.
Scientific claim	A validated emergency-network design	The defensible current claim is narrower: an uncalibrated planning and sensitivity framework plus a field-acquisition system awaiting controlled empirical validation.

The Trial 1 project report documents why Trial 1 is not calibration evidence. The replacement analysis plan defines the model-only simulator work that may proceed before Trial 2.

2. Simulator architecture

2.1 Two-engine design

The simulator has two implementations:

- scripts/mesh_sim.py is the SimPy reference and trace-producing engine. It emits detailed event traces and also contains two Python-only learned modes, q_routing and rl_duty.
- fastsim is a summary-only Rust event engine for repeated seeds, long horizons, and parameter sweeps. It implements the nine shared modes: flood, min_hop, etx, energy_aware, lb_energy, duty_sync, duty_adaptive, rotate_lb, and selective_duty.

Both are discrete-event models, not emulators. They use a half-open simulation interval from time zero up to, but not including, the declared horizon. FastSim schedules exogenous traffic, MAC attempts, transmission completion, rebroadcasts, mobility dispatch/return, energy integration, outages, inventory movement, and SOS retry events. FastSim uses a time-ordered binary heap with a deterministic sequence tie-break.

The intended shared inputs are:

- a fixed-site graph with coordinates, power type, gateway flag, horizon mask, and fixed-link q50 loss;
- timestamped route geometry with q50 loss from each moving route sample to each candidate fixed site;
- daily weather values containing clearness index and snow derating; and
- the central simulation configuration.

FastSim now validates dates, array lengths, coordinates, endpoint identities, link uniqueness, physical ranges, weather duration, timing intervals, radio parameters, battery/solar values, kiosk capacity, and output/input path aliasing before starting. It writes the summary atomically.

2.2 Randomness and event comparability

FastSim randomness is separated by phenomenon and entity rather than drawn from one global stream. Traffic, incident occurrence/time/sender, MAC busy backoff, DIFS, rebroadcast, CAD phase, PHY fading, mobility, and latency sampling do not intentionally consume one another's draws. Traffic arrival clocks are exogenous: a congested MAC does not delay the next offered arrival and thereby change the offered workload.

Within FastSim, the fixed adjacency and output ledgers are sorted before they can influence tie-breaking or serialized output. Same inputs and seed are now expected to produce the same summary bytes across processes. Python and Rust use independent keyed implementations, so cross-engine traces are not expected to be identical.

2.3 Propagation and reception model

For a transmission from a to b at time t, the modeled received power is:

$$\mathrm{Pr}(a,b,t) = 26.30\ \mathrm{dBm} - \mathrm{Lq50}(a,b,t) + \mathrm{Xslow}(a,b,t) + \mathrm{Xfast}$$

The 26.30 dBm reference is the corrected sum of 24.15 dBm transmitter EIRP, +2.15 dBi receive-antenna gain, and the currently assumed 0 dB receive-feed loss. It is not itself EIRP.

Loss is selected as follows:

- **Fixed to fixed:** the topology's precomputed Longley–Rice/ITM median q50 basic loss.
- **Moving radio to fixed site:** linear interpolation in time over the route's precomputed q50 site-loss table.
- **Moving radio to moving radio:** an explicit heuristic: free-space loss at the configured frequency (currently 915 MHz), plus 20 dB clutter, plus 12 dB/km beyond 1.5 km, with an 8 km hard cutoff. This is a workload model, not an empirically fitted hiker-to-hiker propagation law.
- **Missing or excluded link:** an effectively dead 300 dB path.

Slow shadowing is an undirected-link Gauss–Markov process with configured standard deviation 8 dB and 30 s coherence. A separate 2 dB per-packet fast term is added. These are uncalibrated planning values.

Reception requires both:

- RSSI at or above −131 dBm; and
- SNR at or above −17.5 dB, with thermal noise computed as $-174 + 10 \log10(\text{BW}) +$ noise figure.

At 250 kHz and 6 dB noise figure, the modeled noise floor is approximately −114.0 dBm. A fixed edge is eligible for routed trees only when its q50 margin is at least 3 dB. The simulator currently uses q50, not q90, route arrays.

2.4 LoRa airtime, MAC, collision, and CAD model

The airtime calculation uses the configured LoRa SF11, 250 kHz bandwidth, coding rate 4/5, 16-symbol preamble, and payload length. The shared-channel MAC is an abstraction of carrier-sense/managed flooding:

- carrier-sense threshold: −124 dBm;
- slot length: 40 ms;
- random DIFS and busy-channel backoff;
- shorter, higher-attempt priority for SOS traffic;
- SNR-scaled flood contention so weaker/farther eligible relays tend to forward before stronger/nearer relays; and
- duplicate suppression that can cancel a pending flood rebroadcast.

Collision resolution uses a frozen overlap snapshot so equal-time event insertion order cannot change the outcome. A receiver cannot receive while its own transmission overlaps the frame. Concurrent co-SF interference destroys a frame unless the desired frame is at least 6 dB stronger than the strongest interferer. A sender is also treated as one half-duplex resource and cannot start two local transmissions at once.

Duty modes use deterministic periodic preamble sampling:

- one-second CAD/sniff period;
- an awake window equal to the node's duty fraction of that second;
- at least two LoRa symbols of preamble overlap required for acquisition; and
- shared phase zero for duty_sync and selective_duty, stable per-node phases for the other Rust duty modes.

FastSim distinguishes two load quantities:

- aggregate offered airtime is the sum of every transmitter's airtime divided by duration, so simultaneous and spatially isolated frames are additive and the value can exceed one; and
- physical occupancy is a per-fixed-site union of intervals above the carrier-sense threshold, including the site's own transmissions, divided by full run duration.

This model does not implement a waveform, adjacent-channel interference, frequency error, hardware-specific CAD timing, exact Meshtastic firmware queues, every firmware retransmission rule, or a complete regulatory duty-cycle mechanism.

2.5 Routing and duty policies

Flood mode uses hop-limited managed flooding. Every SOS is flooded even when ordinary traffic uses a routed mode.

The deterministic routed modes build a multi-source shortest-path tree rooted at all live MQTT gateways. Mobile radios attach to a reachable fixed site and then use that site's tree path. With optional regional channels, attachment is restricted to the mobile radio's channel component. Tree state is refreshed on the configured interval and, in repaired FastSim duty modes, immediately after availability-changing events.

The edge costs are:

- **min_hop:** one per usable edge;
- **etx:** 1 / modeled success probability, where the probability comes from q50 margin and the assumed lognormal shadowing distribution;
- **energy_aware:** a fixed 0.5-second nominal transmit-energy proxy multiplied by the gatewayward relay's battery/forecast scarcity;
- **lb_energy:** energy_aware cost multiplied by penalties for above-median forwarding EWMA and prior death score; and
- **duty modes:** the lb_energy tree plus a duty assignment.

The scarcity term uses stored energy plus a routing-only estimate of solar generation remaining until midnight. Realized future weather is deliberately not exposed to routing.

Rust duty assignments are:

- duty_sync: non-grid, non-gateway fixed relays at 5%;
- duty_adaptive: 2% to 25% according to energy runway;
- rotate_lb: route-tree relays awake, off-tree relays at 2%; and
- selective_duty: every current route-tree parent and every gateway-critical articulation relay awake, other eligible relays at 5%.

Selective duty does not guarantee an always-awake first hop from a moving radio. A cost-selected ingress leaf that is not otherwise on the awake backbone remains CAD-gated at 5%. That is an explicit reliability/energy trade, not guaranteed end-to-end connectivity.

The Python-only q_routing mode learns experienced next-hop costs; rl_duty learns among 2%, 5%, 10%, and 25% duty actions. These remain exploratory policies, not field-tested controls.

2.6 Energy, battery, outage, and availability model

The nominal portable battery is 37 Wh with 85% usable fraction. For a radio that is not docked or grid powered, baseline drain is:

$$\text{duty} \times \text{listen power} + (1 - \text{duty}) \times \text{light-sleep power}$$

Moving radios also incur the configured GPS current. FastSim charges a transmission only for the incremental current above the actual current baseline during that frame. The default current values—245 mA TX, 68 mA listen, 12 mA light sleep, and 25 mA GPS at 3.7 V—are marked BENCH-CALIBRATE and are not project hardware measurements.

Solar and kiosk charging add energy subject to battery headroom. FastSim now records ordered listen, sleep, GPS, and docked segments with their source kiosk, so a checkout, return, shuttle move, or duty change is not applied backward over the preceding integration interval. A non-grid node becomes unavailable at zero stored energy and may revive at 5% state of charge. Forced outage and energy depletion are distinct states. Availability is integrated as unavailable duration, while death events separately count depletion transitions. FastSim also invalidates a frame if its sender or receiver dies, enters an outage, docks, or rechecks out during the frame.

The repaired Rust engine integrates the fractional final energy interval rather than silently dropping the run tail. It applies a 600 s logical packet TTL so queued or forwarded work cannot transmit indefinitely after its accounting record has been settled. This TTL is an explicit modeling assumption that still needs protocol justification.

2.7 Solar and weather model

The solar model in scripts/solar_model.py, mirrored in Rust, contains:

1. NOAA-approximation sun elevation and azimuth.

2. Haurwitz clear-sky global horizontal irradiance.
3. A direct/diffuse split based on clearness index.
4. Beam shading against a 48-azimuth DEM horizon mask.
5. Canopy transmittance of 0.15 below the modeled treeline threshold.
6. A four-face 35-degree pyramid panel orientation.
7. A 6 W nominal relay panel and 75% system efficiency.

The physical energy process uses each weather day's realized clearness index and snow factor. If Python is run without a weather file, it samples daily clearness from a beta distribution around monthly means. Routing does not use the realized future weather; it uses monthly climatology. Kiosks instead use a flat 200 W array and a 1 kWh energy bank.

Weather-dependent results are valid only when the exact weather product, request, response, date coverage, and hashes are pinned. Older runs labeled ERA5 did not meet that standard. The current daily series is one Mt. Washington-area ERA5/Open-Meteo-derived series applied statewide, with derived clearness and heuristic snow derating; it is not site-specific weather.

2.8 Mobility, rental logistics, and traffic

The shared statewide workload uses timestamped trail routes. Position is linearly interpolated between route samples. FastSim stores the absolute checkout time, so a long walk continues correctly across a UTC day boundary rather than wrapping to the route start at midnight. Radios are off while docked, checked out across a route's scheduled duration, returned to the declared end kiosk, and redistributed by a nightly shuttle.

The kiosk model has:

- 20 radio bays;
- tiered nominal demand of 20, 10, or 4 walkers by route;
- checkouts spread from 11:00 to 17:00 simulation UTC;
- highest-charge eligible-radio selection;
- a 20% minimum checkout state of charge;
- configurable spares, capped by physical capacity;
- 10 W per-bay charging; and
- explicit no-stock and unserviceable-stock starvation counts.

The traffic model includes fixed-site telemetry, moving-radio position beacons, hiker-to-family traffic, paired hiker direct messages, and synthetic SOS incidents. Traffic clocks include keyed jitter and are independent of MAC service. The synthetic SOS process attempts an incident on approximately half of simulation days, sends logical duplicates at incident time +30 s and +60 s, and can issue fresh packet-ID retries every five minutes until a gateway ACK or the retry limit. The assumed incident rate and message behavior are scenario inputs, not measured statewide demand.

Python can additionally replay the legacy Trial 1 hiker track and emit detailed traces. FastSim implements the kiosk-pool summary workload used for sweeps.

3. Why a purpose-specific simulator instead of ns-3

This is an ex-post engineering rationale, not a contemporaneous repository record of an ns-3 selection study. A purpose-specific simulator is justified for the project's present question, but only with a narrow claim.

Advantages

- The principal research variables are not just packet forwarding. They include ITM/DEM loss tables, trail interpolation, daily weather, solar horizon masks, battery death/revival, kiosk stock, charging banks, shuttle logistics, and route-popularity traffic. These are first-class objects in the current model.
- The summary engine can run long horizons and multiple seeds without writing enormous packet traces.
- Keyed randomness makes paired algorithm workloads and debugging easier.
- The model can directly report project-specific outcomes: route delivery, SOS incident delivery/latency, duty misses, unavailable time, relay-energy inequality, checkout service, and receiver-local occupancy.

- A trace-oriented Python implementation and an independently implemented Rust engine provide a useful opportunity to detect coding divergence.

Limitations and the role ns-3 could still play

The custom simulator has much less external validation and protocol depth than a mature general network simulator. It encodes project assumptions directly, so shared mistakes can survive in both engines. It does not prove compatibility with stock Meshtastic or actual radio firmware, and it simplifies the PHY, carrier sensing, capture, CAD, hidden terminals, buffering, retransmission, clock drift, and hardware behavior.

ns-3 would provide a mature event and packet-network framework, standardized tracing, and a broader ecosystem for protocol-level verification. It would still require substantial custom work to reproduce this project's exact LoRa profile, Meshtastic-like flooding, terrain-loss tables, solar/weather model, kiosk logistics, mobility, and project metrics. Moving everything into ns-3 would not itself create field calibration.

The defensible division of labor is therefore:

- use the purpose-specific model for NH planning, sensitivity studies, and whole-year scenario exploration;
- use controlled analytical cases, cross-engine tests, and selected ns-3 or hardware-in-the-loop scenarios for MAC/protocol fidelity; and
- use Trial 2 plus bench measurements for empirical calibration.

The purpose-specific simulator is a planning instrument, not a substitute for firmware testing, ns-3-level protocol studies, or field evidence.

4. Python/Rust cross-validation

4.1 Four different questions

Question	Present answer
Does FastSim pass its current software regressions?	Yes: 41 Rust unit tests and 3 integration tests pass; formatting and strict Clippy are clean.
Is one FastSim seed reproducible across processes?	Yes in the tested case. Four independent 1.3-day min_hop runs with seed 4242 and the locked NH inputs produced identical 423,626-byte JSON and SHA-256 `ed5d23d605e0579f273b85af686030df9cbae2ebcc69f289519abb6da6`

Question	Present answer
Do the current Python and Rust engines agree within preregistered tolerances?	**Yes at micro-scenario scale.** After reconciling seven semantic gaps in `mesh_sim.py` (route-track timing, duty-miss aggregation, selective-duty route-parent wake, global forward-event EWMA, duty-weighted TX energy, horizon truncation, initial duty assignment at t=0), a shared pilot micro-scenario (`scripts/sim_micro_parity.py`, modes flood/duty_sync/selective_duty, seed 42) agrees within every pre-registered tolerance band on PDR, deaths, offered airtime, SOS, and duty-misses — e.g. selective-duty offered-airtime divergence fell from 13.9% to 1.5% and its PDR gap to 0.03%. Three gaps remain deferred and documented (§4.3): ordered energy-segment integration (bounded < 60 s/transition; parity passes without it), a 600 s packet TTL (unreachable under the current hop-limit/CSMA parameters), and mid-frame availability-epoch invalidation (reachable only on sub-second transitions). The CAD wake-phase RNG is documented as engine-specific by design. A **statewide-scale** two-engine parity is still not run because the Python engine is too slow for full-year multi-seed sweeps (~48 s per pilot-day); the micro-scenario is the auditor-endorsed substitute.
Does either engine agree with the real NH system?	Unknown. There is no controlled Trial 2 dataset.

4.2 Historical cross-engine results—retained but withdrawn

The old nine-pair comparison used the retained Rust xval artifacts and Python kiosk summaries. Its mode set predated selective_duty and included lb_energy_r. The following maxima were recomputed from those files; they describe old engine agreement only:

Metric	Maximum old discrepancy	Pair producing the maximum
PDR	0.0057 absolute, historically rounded to 0.006; 0.67% relative	rotate_lb: Python 0.8519, Rust 0.8576
channel_utilization, then used as offered airtime	0.00269 absolute; 2.58% relative	rotate_lb: 0.10143 vs 0.10412
solar-node death events	285 absolute; 3.87% relative	rotate_lb: 7,362 vs 7,077; five of nine pairs were exactly equal
relay TX-energy Gini	0.0065 absolute; 2.01% relative	rotate_lb: 0.3167 vs 0.3232
SOS incidents sent	32 absolute at duty_sync (206 vs 174; 15.53%); maximum relative gap 15.63% at rotate_lb (192 vs 162)	duty_sync / rotate_lb
SOS incidents delivered	30 absolute; maximum relative difference 15.6%	absolute tie at duty_sync, 203 vs 173, and rotate_lb, 192 vs 162; rotate_lb has the larger relative difference

These values are **not current validation results**. They were produced before corrections to collision symmetry, half-duplex handling, CAD misses, traffic RNG isolation, availability accounting, weather provenance, RF metadata, offered-airtime semantics, kiosk constraints, packet lifetime, deterministic ordering, and other audited behavior. Agreement between two implementations also cannot validate a shared model assumption.

4.3 Corrected comparison plan and present status

The prospective replacement analysis plan labels itself pre-registered, but it is currently untracked and is not an immutable external preregistration. It specifies the nine shared modes, seeds 42–46, explicit inputs, and these pass bands:

Metric	Absolute tolerance	Relative tolerance
pdr_overall	0.03	0.05
aggregate_offered_airtime_ratio	0.03	0.10
channel_utilization alias	0.03	0.10
sos.sent	3	0.10
sos.delivered	3	0.10
fleet_energy.deaths_total	none	0.15
duty_misses_total	none	0.15

Within-engine analyses also require seed-level t-based 95% confidence intervals for PDR, offered airtime, fleet energy/availability, SOS delivery and latency, and duty misses. Receiver-local occupancy is diagnostic rather than a parity decision metric.

There are 45 corrected-labeled Rust sweep files but zero corrected Python summary files. Those Rust files also predate the latest FastSim repairs, so they must be regenerated. A current maximum Python/Rust discrepancy therefore does not exist and must not be inferred from the historical 0.006 value.

Before corrected parity can be run, Python must be reconciled with the current Rust semantics. Known gaps include initial duty assignment, selective-duty route-parent wake state, forwarding-load decay, duty-aware incremental TX energy, state-transition energy accounting, overnight route timing, the half-open run horizon, packet TTL, in-flight availability transitions, and the default route refresh interval. These gaps are implementation work, not statistical noise.

5. Main findings: what is now known that was not known at the start

1. **Trial 1 tested the collection system, not the propagation model.** It exposed the need for a controlled source, authenticated sequence denominators, direct-hop filtering, independent geometry, and continuous logging. That is a useful negative finding because it changed Trial 2's design.
2. **The early free-space coverage story was not robust to terrain modeling.** For the Ammo/Jewell concept, later ITM screening reversed important link expectations and produced an approximately 60 dB FSPL-versus-ITM disagreement that Trial 2 can directly test.
3. **The current statewide planning topology does not pass its controlling engineering screen.** With 52 uncalibrated short-link substitutions excluded, the −100 dBm q50 screen reports 87 of 217 sites stranded and 15 of 25 routes below 85% modeled coverage. This is an internal model failure, not measured field coverage.
4. **The old architecture winner is no longer supported.** Collision, duty-wake, traffic-clock, utilization, energy, weather, and parity defects can materially change comparisons. No routing or duty mode should be called best until corrected multi-seed runs and parity accounting are complete.
5. **Reproducibility defects were real and repairable.** FastSim previously changed outputs across separate processes with identical inputs and seed because randomized map iteration affected equal-cost route ties. Canonical ordering and removal of wall-clock timing from scientific JSON now make the tested output byte-identical.
6. **Simulator agreement is a software check, not empirical validation.** This distinction is now explicit in the analysis plan and summary schema.
7. **The dominant uncertainty is empirical.** More simulation runs cannot identify the true receiver threshold, feed/body/foliage losses, shadow variance, board current, panel yield, CAD behavior, or real traffic. Controlled field and bench data are now the critical path.

5A. Corrected multi-seed results (2026-07-14)

The corrected FastSim engine was run for all nine shared modes across seeds 42–46 (365 days, NH statewide, hash-locked inputs frozen at `artifacts/sim/corrected/release_v1/`, aggregated in `corrected_stats.json`). These are **MODEL-ONLY, uncalibrated** results. The Python arm is not yet run at statewide scale — bucket-3 reconciliation is at the micro-scenario stage, where the two engines agree within the pre-registered tolerances on PDR, deaths, and duty-misses (§4). So these are single-engine numbers cross-checked at micro-scale, **not** a completed statewide two-engine parity.

Mode	PDR (95% CI)	Death-events/yr	SOS delivered	duty-misses/yr	Class
flood	0.911 ± 0.000	29,234	189/189 (100%)	0	always-on
min_hop	0.862 ± 0.000	29,164	189/189 (100%)	0	always-on
etx	0.866 ± 0.000	29,165	189/189 (100%)	0	always-on
energy_aware	0.866 ± 0.000	29,164	189/189 (100%)	0	always-on
lb_energy	0.866 ± 0.000	29,164	189/189 (99.9%)	0	always-on
duty_sync	0.728 ± 0.000	1,217	179/189 (94.7%)	23.8 M	duty
duty_adaptive	0.755 ± 0.003	1,737	188/189 (99.3%)	25.6 M	duty
rotate_lb	0.809 ± 0.000	4,877	188/189 (99.5%)	24.1 M	duty
selective_duty	0.811 ± 0.000	5,311	188/189 (99.3%)	23.3 M	duty

Observed tradeoffs — **no single mode is declared best** (full statewide parity is still pending, per §9):

- Always-on modes hold PDR 0.86–0.91 but incur ~29,000 death-events/yr: solar relays cycle depleted/revived through the winter energy deficit.
- Duty-cycled modes cut death-events 5.5–24× but lower PDR to 0.73–0.81. This is the survival-versus-delivery tradeoff, now quantified on the corrected engine.
- SOS (the life-safety class, flooded in every mode) is recovered to ~99% by `duty_adaptive`, `rotate_lb`, and `selective_duty` — each keeps energy-rich or connectivity-critical relays awake — while uniform `duty_sync` drops SOS to 94.7%.
- Among the ~99%-SOS duty modes, `duty_adaptive` reaches it at the fewest deaths (1,737); `selective_duty` reaches it at higher PDR (0.811) but more deaths (5,311), because keeping the connectivity-critical backbone always awake drains those nodes. No mode dominates on all of PDR, survival, and SOS simultaneously; the choice is an explicit weighting decision, which is why no "winner" is named.

Confidence intervals are tiny (< 0.005 on PDR) because the engine is near-deterministic given a seed — traffic and weather are seed-fixed, so seed variation is not the dominant uncertainty. The dominant uncertainty remains **empirical** (uncalibrated propagation and energy parameters), per §7, not statistical.

5A.1 Decision-oriented metrics

Raw counts answer "what happened in the simulator"; the following metrics (`scripts/build_meaningful_metrics.py` → `artifacts/sim/corrected/release_v1/meaningful_metrics.json`) answer the questions a SAR planner would ask. Same runs, same caveats (MODEL-ONLY).

Mode	SOS p95 latency	Fleet availability	Site-years dark/yr	Sites <90% avail	mWh/delivered pkt
flood	32 s	53.5%	74.0	118	1.44
min_hop	5 s	53.7%	73.6	118	0.42
etx	31 s	53.7%	73.6	118	0.42
energy_aware	4 s	53.7%	73.6	118	0.42
lb_energy	5 s	53.7%	73.6	118	0.42
duty_sync	**35 min**	**96.5%**	**5.6**	**7**	**0.30**
duty_adaptive	15 min	94.7%	8.5	13	0.40
rotate_lb	**33 s**	91.0%	14.3	31	0.35
selective_duty	**62 s**	89.7%	16.5	33	0.36

Two things the raw counts hid:

1. **Death-event counts understated the always-on problem.** The always-on modes leave ~46% of solar fleet-time dark (118 of ~159 solar sites are individually available less than 90% of the year). "29,000 death events" reads like churn; "half the solar network is down at any given time" is the operational reality of the uncalibrated model.
2. **SOS delivery percentage hid a latency cliff.** Uniform `duty_sync` delivers 94.7% of SOS incidents — but at a 35-minute 95th-percentile latency, because deliveries ride the 5-minute retry loop. `rotate_lb` and `selective_duty` deliver ~99.4% at 33–62 s p95 by keeping a relay backbone awake. Every mode retains rare 1–2 hour worst-case incidents (the retry ceiling), so the maximum is a shared protocol limitation rather than a differentiator. For a life-safety network, tail latency — not delivered fraction alone — is the operative SOS metric, and it reorders the duty family: the backbone-awake variants dominate uniform sleep on every SOS measure while conceding availability.

No overall winner is declared for the same reason as above: the three axes (delivery, availability, SOS tail latency) still trade off, and the corrected model's role is to expose the envelope, not to pick the operating point.

6. Important mistakes and corrected claim categories

The audit found mistakes, not evidence of intentional lying:

- collision and half-duplex outcomes could depend on event/insertion order;
- a receiver's duty cycle reduced energy without always creating a corresponding reception miss;
- a global offered-airtime sum was called physical channel utilization;
- traffic and policy RNG consumption could change workloads across algorithms;
- deaths, unavailable duration, latency sampling, SOS retries, and kiosk service could be miscounted;
- Rust same-seed runs could differ across processes;
- RF metadata mislabeled a TX+RX link-budget reference as EIRP;
- historical weather files were called ERA5 without a pinned ERA5 request and sufficient provenance;
- uncalibrated short-link substitutions helped produce a misleading statewide PASS; and
- zero-eligible Trial 1 outputs were described too much like calibration evidence.

The ledger records which code paths are fixed and which scientific results remain withdrawn. "Fixed" means corrected in the current worktree; it does not mean merged, independently reviewed, rerun at full scale, field-validated, or released.

7. Limitations

Empirical limitations

- There is no controlled Trial 2 empirical dataset.
- Trial 1 currently has zero calibration-grade joined observations.
- The project has no held-out field estimate of RSSI error, PDR error, shadow variance, capture behavior, or spatial/temporal transfer.
- Feed loss, antenna installation, body loss, foliage loss, receiver behavior, TX power, RX/listen/sleep currents, cold battery capacity, and panel yield are not measured for the exact intended hardware configuration.
- Trial 2 can calibrate propagation and direct-link PDR on its sampled routes. It cannot by itself validate statewide traffic, routing, failure, kiosk, or annual energy behavior.

Modeling limitations

- ITM q50 is used as a median terrain-loss prior; route-level q90 arrays and a validated residual model are absent.
- The 8 dB slow-shadow, 2 dB fast-fade, 30 s coherence, −131 dBm receiver threshold, zero feed loss, capture threshold, and mobile-mobile heuristic are assumptions.
- Site placement, antenna heights, permissions, maintenance, vandalism, weatherization, and installation feasibility are outside the network simulation.
- Solar uses gridded/weather-model inputs and simplified canopy, snow, horizon, and panel geometry rather than measured site irradiance and hardware curves.

- Synthetic traffic and SOS rates are scenario choices, not observed demand.
- The MAC is not stock firmware and the routed/duty modes may require a custom companion or firmware architecture.
- Security, privacy, custody, authenticated command completion, human factors, and incident-command operations are not validated by packet delivery.

Software and release limitations

- Python and Rust are not yet semantically reconciled after the latest repairs.
- The current worktree is dirty and concurrently edited; no immutable release binds this report to a reviewed commit and artifact manifest.
- Current corrected-labeled Rust artifacts are stale relative to FastSim code.
- Some source comments outside the repaired engine still repeat superseded claims—for example, the hardware-current comment in the simulation YAML says conclusions were "verified invariant" even though those runs are withdrawn.
- Dependency advisory scanning for Rust was unavailable in the local toolchain during this audit.
- Passing regression tests does not establish absence of defects. A transmitter that loses availability mid-frame invalidates delivery, but its already-booked frame remains in the simplified channel-occupancy/interference model until nominal frame end; this needs a dedicated PHY-abort model.

8. Future work: how Trial 2 should validate or recalibrate the simulator

Trial 2 should be used as a prospective test of a frozen model, followed by a separate recalibration and held-out evaluation:

1. **Freeze before collection.** Version the exact hardware, firmware, channel, bandwidth, TX power, antenna/feed arrangement, beacon cadence, route, strata, topology, DEM, predictions, exclusions, scripts, and hashes. Record amendments rather than overwriting the frozen pack.
2. **Create a real denominator.** Use one surveyed static beacon per field day, a fixed 30 s cadence, monotonic authenticated sequence numbers, synchronized clocks, and direct-packet eligibility. Missing sequences remain failed opportunities.
3. **Measure geometry and context.** Retain receiver GNSS/time, antenna height and orientation, body placement, direction of travel, canopy/terrain stratum, weather, battery state, and every exclusion.
4. **Replicate by pass/day.** Target at least 40 scheduled opportunities per primary stratum across independent passes, but analyze whole passes or time blocks rather than treating correlated packets as independent replicates.
5. **Score the frozen model first.** Report per-stratum measured RSSI and direct PDR, scheduled denominators, dependence-aware uncertainty, RSSI residuals, and out-of-sample RMSE. Apply the prospective ± 12 dB RSSI and ± 0.15 PDR engineering screens as diagnostics, not equivalence tests.
6. **Recalibrate parsimoniously.** On training passes only, estimate a small, identifiable set such as global link-budget offset, shadow standard deviation/coherence, effective threshold, and justified foliage/body offsets. Do not fit enough parameters to explain every sampled stratum.
7. **Evaluate on held-out passes or days.** Compare frozen and recalibrated models on untouched data. Report error and calibration plots even if the model fails. If transfer is poor, revise the propagation structure rather than declaring success from in-sample fit.
8. **Propagate accepted calibration.** Version a new calibration file, regenerate q50/q90 route-loss tables, run both engines from the same locked inputs, and repeat parity and multi-seed analyses.
9. **Validate the other layers separately.** Bench-measure current and battery behavior; use hardware-in-the-loop experiments for CAD, capture, hidden terminals, retransmission, and firmware queues. Trial 2 propagation data must not be stretched into validation of those layers.

The current Trial 2 preregistration and runbook are the starting point, but the pack remains explicitly not frozen.

9. Collective priorities

1. Let concurrent file work finish, then review and integrate changes in small, attributable commits; do not generate evidence from the mixed worktree.

2. Port the repaired FastSim semantics to Python, add matched micro-scenario tests, and resolve every declared cross-engine difference.
3. Rebuild both engines from a clean commit; run the nine shared modes for seeds 42–46 with locked inputs; emit manifests and hashes; run the parity report; investigate every tolerance flag.
4. Do not interpret algorithm rankings until step 3 is complete.
5. Freeze and execute the controlled NH Trial 2 protocol, preserving failures and underpowered strata.
6. Complete exact-hardware RF and power bench measurements plus targeted hardware-in-the-loop MAC tests.
7. Recalibrate only on training data, evaluate on held-out passes/days, then regenerate the simulator study and final report.

Until those steps close, the correct project description is: **a New Hampshire research prototype and uncalibrated planning study with a repaired but not yet cross-validated or field-calibrated simulator**.